

Project: Automated Access Control List (ACL) Updates

Role: Security Professional

Company Domain: Health Care

1. Project Description

In this scenario, I am responsible for maintaining the security of a restricted subnetwork containing sensitive patient records. Access to this content is managed through an allow list of IP addresses. My task was to develop a Python algorithm that automates the process of checking this allow list against a remove list and programmatically purging unauthorized IP addresses to ensure only permitted employees have access.

2. Open the file that contains the allow list

The first step of the algorithm is to access the data. I assigned the filename `allow_list.txt` to the variable `import_file`. I then used a `with` statement and the `open()` function with the `"r"` parameter to open the file in read mode. Using the `with` keyword is a best practice because it ensures the file is closed automatically after the code finishes running.

3. Read the file contents

Once the file was open, I utilized the `.read()` method to convert the contents of the file into a string format. I stored this string in a variable named `ip_addresses` so that the data could be manipulated in later steps.

4. Convert the string into a list

To remove individual IP addresses, the data needed to be in a list format rather than a single string. I used the `.split()` method to break the string into separate elements, allowing the algorithm to iterate through each address individually.

5. Iterate through the remove list

I implemented a `for` loop to check every IP address that was scheduled for removal. By using `element` as the loop variable, the algorithm cross-references each unauthorized IP against the main `ip_addresses` list.

6. Remove IP addresses that are on the remove list

Inside the loop, I created a conditional if statement to check if the current element was present in the `ip_addresses` list. If a match was found, I applied the `.remove()` method to delete that specific IP from the allow list.

7. Update the file with the revised list of IP addresses

To save the changes, I first converted the list back into a string using the `.join()` method, applying it to a newline character (`\n`) to maintain the file's original format. Finally, I used a `with` statement and the `open()` function with the `"w"` (write) parameter to overwrite `allow_list.txt` with the updated string.

8. Summary

This algorithm provides an automated and reliable way to manage access control for sensitive data. By utilizing Python's file handling and list methods, I created a script that can be easily updated and reused for different security logs. This automation reduces the risk of human error and ensures that unauthorized access is revoked immediately across large datasets.